

算法 / 开发工程师 Git 与 GitHub 协作编码标准流程手册

从 0 到 1 上手，到专业团队协作：分支、Issue、PR、review、CI、merge queue、release、回滚、contributor 与工程健康门禁。

手册定位

这是一套面向工程交付的 SOP，而不是命令速查表。它把 Git 的本地历史管理、GitHub 的协作机制、Code Review 的工程标准和开源 contributor 流程放在同一个闭环里。

参考对象只取公开可验证资料：Git/GitHub 官方文档、Google Engineering Practices、Meta Engineering、OpenAI 官方仓库贡献说明、CloudWeGo / ByteDance 开源项目贡献说明，以及 SemVer、Conventional Commits 等公开规范。

Winston

资料口径截至 2026-05-30

Git · GitHub · Google · Meta · OpenAI · CloudWeGo

00 | 目录与阅读路径

先学闭环
再学命令

01	总原则：专业协作不是会用 Git	目标与质量观	13	外部 Contributor 流程	fork · maintainer
02	Git 心智模型	worktree · index · commit	14	算法工程特殊规范	data · model · experiment
03	从 0 到 1 环境准备	config · SSH · gh	15	恢复与排障	reflog · revert · bisect
04	日常开发循环	status · diff · commit	16	高级 Git 操作	worktree · cherry-pick · patch
05	分支策略	trunk · GitHub Flow · release	17	安全与合规	secret · signing · access
06	Issue 工作流	intake · triage · planning	18	AI 协作编码边界	agent · generated code
07	小变更与提交规范	atomic · conventional	19	角色、RACI 与团队制度	author · owner · release
08	同步、rebase 与冲突	fetch · merge · rebase	20	模板与 Checklist	issue · PR · review
09	Pull Request 标准流程	draft · template · CI	21	命令速查	高频与危险命令
10	Code Review 标准	author · reviewer	22	成熟度评分	团队自检
11	分支保护与 merge queue	required checks	23	来源与边界	公开资料口径
12	合并、发布与回滚	SemVer · tag · changelog			

READING CONTRACT

如果只想立刻能协作，读 01 到 12。如果你是开源维护者，追加 13、17、19、20。如果你做算法、推荐、搜索、LLM 或数据密集型研发，必须读 14。遇到事故或历史错乱，先读 15，再碰危险命令。

01 | 总原则：专业协作不是会用 Git

专业工程师使用 Git/GitHub 的目标，不是把代码推上去，而是让每一次变更都有清楚的目的、足够小的审查面、可重复的验证证据、明确的责任人和可执行的回滚路径。

PRINCIPLE 01

小而完整

每个 PR 只解决一个问题，包含必要测试和文档。Google 的公开 code review 文档强调小变更更快、更彻底、更易回滚。

PRINCIPLE 02

事实优先

争议按数据、测试、规范、现有架构和用户影响判断。纯偏好不应阻塞合并，风格以项目 style guide 为准。

PRINCIPLE 03

主干健康

默认分支始终可构建、可测试、可发布。保护主干比保护个人分支重要，所有合并都要经过自动化和人工门禁。

PRINCIPLE 04

证据闭环

PR 描述、CI、review、commit、release notes 和 issue link 共同构成工程证据链。没有证据，不叫完成。

大厂公开实践抽象成四条团队规范

公开实践来源	可迁移原则	落地到 GitHub
Google Engineering Practices	Code review 服务于长期 code health，小变更、测试和清晰命名优先。	PR 小于一个概念变更，review checklist 覆盖设计、功能、复杂度、测试、文档。
Meta Engineering	大规模仓库需要高信号工具链、可恢复历史和 stacked changes 支撑并行开发。	允许 stacked PR，但每层必须独立可审查；用 CI 和 merge queue 降低主干破坏概率。
OpenAI 官方仓库	贡献说明要明确环境、生成代码边界、测试、lint 和发布管线。	CONTRIBUTING.md 写清 bootstrap、test、lint、generated files、release automation。
CloudWeGo / ByteDance 开源项目	先 issue 后大改，PR 要有测试、commit 约定、清晰标题和安全报告路径。	Issue 模板、PR 模板、labels、security policy、维护者 triage 规则进入仓库。

PROFESSIONAL BAR

一名合格工程师会写代码；一名专业协作工程师能解释为什么这个改动属于这个 issue、为什么拆成这个 PR、为什么这些测试足够、为什么现在可以合并、合并失败时怎么退回。

02

Git 心智模型：先理解对象，再记命令

大多数 Git 事故
来自把层级混在一起

Git 的日常操作围绕四层状态展开：工作区、暂存区、本地仓库、远端仓库。理解这四层，才能判断一个命令是在改文件、改下一次提交、改本地历史，还是改远端共享历史。

WORKING TREE

工作区

你正在编辑的文件。git status 显示它相对 index 和 HEAD 的差异。

INDEX

暂存区

下一次 commit 的候选快照。git add 改 index，不是“上传”。

LOCAL REPO

本地提交图

commit、branch、tag、HEAD 都在这里。
commit 只写本地历史。

REMOTE

远端引用

origin/main 是你上次 fetch 看到的远端状态，不等于实时 GitHub 状态。

关键对象

对象	含义	常见误区	检查命令
HEAD	当前检出的提交。	以为 HEAD 一定等于分支名。detached HEAD 时不是。	git rev-parse --abbrev-ref HEAD
branch	指向某个 commit 的可移动名字。	以为 branch 存一份代码副本。它只是引用。	git branch -vv
commit	一次完整快照，含父提交、作者、消息。	以为 commit 是文件 diff。diff 是两个快照的比较结果。	git show --stat
tag	指向特定提交的发布标记。	发布后改 tag，会破坏可追溯性。	git tag --sort=-version:refname
upstream	本地分支跟踪的远端分支。	pull / push 目标不清。	git status -sb

SAFETY RULE

会重写历史的命令只允许作用在自己的未合并分支上。凡是会影响远端共享历史的动作，先确认分支、上游、PR 状态和团队约定。

03 | 从 0 到 1 环境准备

先让身份、认证、远端
全部可解释

初学者最容易跳过身份和认证配置，后续会在 commit 作者、push 权限、SSH key、多个账号和 GPG 签名上反复出错。准备阶段的目标，是让本机、GitHub 账号、仓库远端和 CLI 都能稳定配合。

一次性配置

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --global pull.ff only
git config --global push.default simple
git config --global fetch.prune true
```

配置	推荐值	原因
init.defaultBranch	main	与 GitHub 默认主分支一致。
pull.ff	only	避免无意识 merge commit。
fetch.prune	true	自动清理远端已删除分支引用。

认证与仓库获取

```
ssh-keygen -t ed25519 -C "you@example.com"
ssh -T git@github.com
gh auth login
gh auth status

git clone git@github.com:OWNER/REPO.git
cd REPO
git remote -v
git status -sb
```

MULTI ACCOUNT

公司账号、个人账号和开源账号分开时，优先用 SSH host alias 或 per-repo user.email。不要在一个仓库里混用多个身份，后续审计和贡献统计都会变乱。

新仓库初始化

```
mkdir my-project && cd my-project
git init
git add README.md
git commit -m "docs: add project readme"
git remote add origin git@github.com:OWNER/my-project.git
git push -u origin main
```

04 | 日常开发循环：每次动手都走同一条线

固定循环
降低认知负担

日常开发的标准循环是：更新基线，创建任务分支，编辑，检查 diff，分块暂存，提交，推送，开 PR。不要在 main 上直接写功能，不要在没看 diff 时 commit，不要把无关格式化混进业务改动。



本地开发纪律

纪律	为什么	违反后的处理
工作树脏时不切任务。	避免把上一个任务的残留带入新 PR。	提交、stash、worktree 或放弃，必须显式选择。
提交前必须看 <code>git diff</code> 。	防止调试日志、密钥、临时文件进入历史。	若已提交但未推送，用 <code>commit --amend</code> 修正。
不要用 <code>git add</code> ，替代思考。	它会把无关文件一起放进 commit。	用 <code>git restore --staged</code> 退回 index。
所有本地验证写入 PR。	reviewer 需要知道你验证了什么。	PR 描述补充命令与结果。

05

| 分支策略: 默认短分支, 必要时 release 分支

分支是隔离工作
不是长期存货

GitHub Flow 的公开文档给出了一条轻量路径: 从默认分支创建分支, 提交, 打开 PR, review, 合并, 删除分支。专业团队通常以短生命周期 feature branch 加 protected main 为默认, 只有在发布节奏、客户支持或兼容窗口需要时才增加 release branch。

DEFAULT

Trunk based + PR

所有变更通过短分支 PR 合入 `main`。适合持续集成、频繁发布和小团队。

WHEN NEEDED

Release Branch

从稳定点切出 `release/1.8`, 只接收修复和发布准备。适合客户端、SDK、企业部署。

AVOID BY DEFAULT

Long-lived Develop

长期 `develop` 会推迟集成风险。只有明确多环境发布制度时才保留。

分支命名标准

类型	格式	例子	使用场景
feature	feat/<issue>--<slug>	feat/128-vector-cache	新增功能。
fix	fix/<issue>--<slug>	fix/233-empty-response	缺陷修复。
docs	docs/<slug>	docs/contributor-guide	文档。
chore	chore/<slug>	chore/ruff-upgrade	工具、依赖、CI。
release	release/<major.minor>	release/2.4	发布稳定分支。
hotfix	hotfix/<version>--<slug>	hotfix/2.4.1-auth-timeout	生产紧急修复。

BRANCH RULE

一个分支只服务一个任务。分支超过 3 到 5 天仍不能开 PR, 通常说明任务拆分失败。先切出最小可合并单元, 再继续后续工作。

06 | Issue 工作流: 把需求入口变成工程事实

Issue 不是留言板
它是变更合同

GitHub Issues 可以跟踪 bug、功能、想法、任务和计划。专业流程的关键，是让 issue 在进入开发前具备可复现、可验收、可排期、可分派的结构，而不是靠 PR 阶段再补上下文。

INTAKE

收集

模板要求背景、现象、期望、复现、影响面、版本、截图或日志。

TRIAGE

分诊

维护者判断类型、优先级、范围、owner、是否需要设计讨论。

PLAN

计划

明确验收标准、非目标、拆分子任务、依赖和目标 milestone。

LINK

闭环

PR 使用 Fixes #123 或 Refs #123 连接 issue。

Label 体系

维度	标签例子	用途
类型	type:bug type:feature type:docs type:refactor	决定处理流程和模板。
优先级	p0 p1 p2 p3	决定排期，不等于严重程度。
状态	needs-info accepted blocked in-progress	让队列可扫描。
新人入口	good first issue help wanted	给外部 contributor 清晰入口。
风险域	security performance breaking data	触发额外 reviewer 或门禁。

TRIAGE RULE

缺少复现信息的 bug 不进入实现；缺少成功标准的 feature 不进入开发；涉及安全漏洞的报告不放在公开 issue 里处理，应转入安全披露渠道。

07 | 小变更与提交规范

commit 给机器和未来的人读
PR 给 reviewer 读

Google 的公开实践强调一个 CL 应当是 self-contained change，并包含相关测试。GitHub Flow 也建议每个 commit 包含独立完整的变化，便于 revert。这里的核心不是“行数崇拜”，而是每个变更单元能被理解、验证、撤销。

原子提交规则

- 一个 commit 只表达一个原因：功能、修复、重构、测试、文档、CI 中的一类。
- 重命名和行为修改分开提交，避免 reviewer 看不清真实 diff。
- 格式化整文件和逻辑修改分开提交。
- 生产代码改变和相关测试应在同一个 PR 中，通常在同一个或相邻 commit 中。
- 提交消息说明“为什么”和“影响”，不写空泛的 update code。

Conventional Commits 推荐格式

```
<type>[optional scope]: <description>

[optional body]

[optional footer(s)]
```

类型与 SemVer 对应

类型	含义	版本影响	例子
fix	缺陷修复。	PATCH	fix(api): handle empty batch result
feat	新增能力。	MINOR	feat(rank): add vector cache
! 或 BREAKING CHANGE	不兼容修改。	MAJOR	feat(api)!: rename search endpoint
docs	文档。	无默认版本影响	docs: add contributor guide
test	测试。	无默认版本影响	test(auth): cover token refresh
ci	CI 配置。	无默认版本影响	ci: require linux smoke test

DANGER

已推送并被别人基于其开发的 commit 不要随意 rewrite。只有在个人 PR 分支、团队允许且使用 --force-with-lease 时，才改远端历史。

08 | 同步、rebase 与冲突处理

同步不是机械 pull
它是在选择历史形状

团队协作中，最常见的历史问题来自混用 pull、merge、rebase 和 force push。默认建议：更新远端状态用 `fetch`，主干更新用 `pull --ff-only`，个人 PR 分支可用 `rebase` 整理，公共分支禁止改写历史。

更新 main

```
git switch main
git fetch origin
git pull --ff-only
```

让个人分支追上 main

```
git switch feat/123-x
git fetch origin
git rebase origin/main
```

安全推送重写后的个人分支

```
git push --force-with-lease
```

merge 与 rebase 的选择

场景	推荐	理由	禁止
更新本地 main	<code>pull --ff-only</code>	main 应保持远端线性一致。	在 main 上产生本地 merge commit。
个人 PR 分支追上 main	<code>rebase origin/main</code>	PR 历史更清楚，冲突提前解决。	多人共享分支上 rebase 后强推。
集成长期 release 分支	按团队策略 merge 或 cherry-pick	需要保留发布历史和修复来源。	无记录地复制代码。
Stacked PR	每层 rebase 到上一层最新提交	保持依赖链可审查。	把多层变化压成一个巨大 PR。

冲突处理 SOP

1. 先读冲突文件上下文，确认两边改动意图。
2. 只解决语义冲突，不盲目接受 ours 或 theirs。
3. 解决后运行最小相关测试。
4. 如果冲突来自架构方向分歧，停下来找 reviewer 或 owner 对齐。
5. rebase 中用 `git rebase --continue`，不能继续时用 `git rebase --abort` 回到起点。

09 | Pull Request 标准流程

PR 是可审查变更包
不是代码上传窗口

GitHub 官方文档把 PR 放在协作反馈和合并门禁中心。专业 PR 应当让 reviewer 不需要猜：改了什么、为什么改、怎么验证、风险在哪里、哪些问题不在本 PR 解决。

DRAFT

早开 draft PR

需要方向反馈、架构讨论或 CI 预跑时，用 draft PR。不要等全部写完才暴露方向风险。

READY

转 ready 的门槛

scope 收敛、测试通过、描述完整、无明显 debug 代码、无 secret、reviewer 能开始判断。

MERGE

合并前条件

必需检查通过、必要 reviewer 批准、conversation resolved、风险和回滚已说明。

PR 描述模板

```
## Summary
- What changed:
- Why now:
- Linked issue: Fixes #123

## Verification
- [ ] unit tests:
- [ ] integration / e2e:
- [ ] manual check:

## Risk
- User impact:
- Rollback:
- Out of scope:

## Reviewer notes
- Files to read first:
- Generated files:
- Migration / release notes:
```

PR 规模门禁

规模	默认处理	review 要求
< 200 行概念 diff	标准 PR。	1 名 owner 或熟悉模块的 reviewer。
200 到 600 行	解释为什么不能再拆。	至少 1 名模块 owner，必要时补设计说明。
> 600 行或跨 5 个模块	优先拆 PR 或 stacked PR。	架构 review、测试证据、回滚方案。
迁移、权限、安全、数据	不按行数判断，直接高风险。	owner、security / infra / data reviewer 和发布门禁。

10 | Code Review 标准: 提高 code health, 而不是找茬

review 的产物
是更健康的主干

Google 的 code review 标准可以压缩成一句工程原则: 当变更整体上明确提升系统 code health 时, reviewer 应倾向批准, 即使它并不完美。专业 review 既不能用个人偏好阻塞进展, 也不能让会降低长期可维护性的改动进入主干。

Reviewer 检查顺序

- 1. **Scope**, 这个 PR 是否解决了声明的问题, 是否混入无关改动。
- 2. **Design**, 模块边界、抽象、依赖方向、数据流是否合理。
- 3. **Functionality**, 功能是否符合用户和开发者需求, 边界条件是否覆盖。
- 4. **Complexity**, 有没有过度设计、难懂控制流、未来需求猜测。
- 5. **Tests**, 测试是否会在代码坏掉时失败, 是否覆盖关键路径。
- 6. **Security / Privacy / Data**, 输入、权限、敏感数据、外部调用是否安全。
- 7. **Documentation**, 构建、使用、发布、配置行为改变是否同步文档。

评论等级	含义	作者义务	Reviewer 义务
Blocking	不修会导致 bug、风险或 code health 下降。	修复或给出可验证反证。	说明触发条件和后果。
Suggestion	更好但不必阻塞。	接受、拒绝或解释。	不要伪装成必须项。
Nit	小风格、命名、排版。	可处理。	不能据此 request changes。
Question	需要理解意图。	优先让代码或文档更清楚。	如果只是学习问题, 标明非阻塞。

Author 回应规则

- 先判断 reviewer 是否指出了真实 code health 问题。
- 能通过改代码澄清的, 不用评论解释替代。
- 不同意时写 tradeoff: 选择 A 的原因、B 的代价、你希望 reviewer 判断的点。
- 对 nit 可选择处理, 但不要让小修饰掩盖重大问题。
- review 来回超过两轮仍卡住, 转同步讨论, 并把结论写回 PR。

COMMENT STYLE

评论应指向具体代码和具体后果。用“这里在空列表时会返回 500, 因为下游没有处理 None”替代“这里感觉不够优雅”。

11 | 分支保护、CODEOWNERS 与 merge queue

主干稳定性
靠制度不是靠自觉

GitHub branch protection 可以要求 PR review、状态检查、conversation resolution、签名提交、线性历史、merge queue 和部署成功。成熟团队应把这些规则固化在默认分支上，让主干健康成为默认路径。

PROTECTION

主干保护

禁止直接 push，要求 PR、至少 1 到 2 个审批、必需状态检查和 conversation resolved。

OWNERSHIP

CODEOWNERS

把关键目录映射到 owner。修改对应文件时自动请求 owner review，并可作为必需审批。

QUEUE

Merge queue

高频合并时，queue 会在最新主干和队列前序 PR 上重新验证，减少“单独都绿，合并后变红”。

推荐规则矩阵

仓库类型	主干规则	CI	Review	补充
个人学习仓库	建议保护 main。	至少 lint 或 test。	可自审。	练习 PR 流程。
团队业务仓库	禁止直推，Require PR。	unit、lint、typecheck、build。	1 owner。	启用 auto delete branch。
核心服务 / SDK	必需检查、签名、线性历史。	跨平台、集成、兼容性。	2 reviewer 或 CODEOWNERS。	发布前 smoke / staging。
高频合并 monorepo	merge queue。	queue context 重新跑。	owner + area specialist。	按目录 owner 和影响面分级。

CODEOWNERS 示例

```
# Default owners
* @org/core-maintainers

# Critical surfaces
.github/ @org/release-engineering
/infra/ @org/infra
/security/ @org/security
/models/ @org/ml-platform
/docs/ @org/docs-maintainers
```

HARD RULE

保护 CODEOWNERS 文件本身。否则攻击者或误操作可以先改 owner，再绕过关键目录审批。

12 | 合并、发布与回滚

发布不是 merge 后的附属动作
它是交付合同的一部分

合并策略决定历史形状，版本策略决定用户如何理解变化，回滚策略决定事故成本。团队应在仓库级别明确 merge method、版本规则、release notes、tag、hotfix 和 rollback。

SQUASH MERGE

默认推荐

把 PR 压成一个语义 commit，适合 GitHub Flow 和外部 contributor。maintainer 可整理 commit message。

REBASE MERGE

线性历史

保留每个 commit，要求作者提交质量高。适合内部团队和精心组织的 commit 链。

MERGE COMMIT

保留分支语义

适合 release branch、长期分支和需要保留集成点的仓库。主干会出现非线性历史。

发布 SOP

- 1. 确认 main 绿，release blocker 为 0。
- 2. 核对版本号：SemVer 中 MAJOR 表示不兼容 API，MINOR 表示向下兼容功能，PATCH 表示向下兼容修复。
- 3. 从 commit / PR / issue 生成 changelog，面向用户写影响，不只是列 commit。
- 4. 创建 tag 和 release，产物、校验和、镜像、包管理器版本保持一致。
- 5. 发布后验证安装、启动、关键路径和回滚命令。

场景	动作	命令	注意
普通回滚	revert PR 或 commit。	<code>git revert <sha></code>	保留历史，适合已推送主干。
热修复	从 release branch 切 hotfix。	<code>git switch -c hotfix/1.8.1-x release/1.8</code>	修复后回合 main。
误发 tag	优先发新版本。	<code>git tag v1.8.1</code>	已公开 tag 不应静默改写。
feature flag 回滚	关闭配置。	按系统配置执行	仍需补代码层修复。

13 | 外部 Contributor 流程

开源协作靠清晰入口
不是靠维护者临场解释

GitHub 官方 contributing guide 以 fork、clone、branch、commit、push、pull request 为基础路径。ByteDance / CloudWeGo 项目贡献说明还强调：先查重 issue / PR，大改先讨论，提交测试，遵守 commit 约定，安全漏洞不要走公开 issue。

角色 / 阶段	找任务	准备环境	开发	PR	Review	合并后
Contributor	读 README、CONTRIBUTING、good first issue。	fork、clone、安装依赖、跑测试。	创建分支，小改动，补测试。	填写模板，链接 issue。	回应评论，更新分支。	删除分支，关注 release。
Maintainer	维护标签、分诊、说明期望。	保证 bootstrap 文档可用。	必要时提前设计评审。	检查范围、CI、许可证。	给可执行反馈。	merge、close issue、release note。

仓库必须提供的 contributor 资产

CONTRIBUTING.md

环境准备、开发命令、测试、lint、PR 规则、generated code 边界、发布规则。

CODE_OF_CONDUCT.md

社区行为边界和升级渠道。大型开源项目建议提供。

SECURITY.md

安全漏洞报告邮箱、响应预期、不要公开 issue 的说明。

Issue / PR Templates

标准化收集复现、动机、验证和风险，减少维护者来回询问。

Maintainer 处理 PR 的规则

- 方向正确但实现不达标，优先帮助 contributor 收敛，而不是静默重写后关闭。
- 需要 maintainer 修改 contributor 分支时，确认 maintainer edits 是否允许。
- 不接受的 PR 要给明确原因：范围不符、已有实现、风险过高、缺少 issue 讨论或违反兼容性。
- 外部贡献者不一定熟悉内部偏好，反馈应落在可执行动作上。

14 | 算法工程特殊规范：代码、数据、模型、实验分开管

算法 PR 最大风险
是不可复现和资产污染

算法、推荐、搜索、LLM、CV 和数据工程的 Git 协作，不只管理代码，还涉及数据口径、特征、配置、模型权重、实验指标和离线 / 在线一致性。默认原则：Git 管代码和小型文本资产，大数据和权重进入专门系统，实验结果要能复现。

CODE

训练 / 推理代码

正常进入 Git，必须有单测、配置示例、最小可运行命令和版本约束。

CONFIG

实验配置

可进入 Git。记录数据集版本、随机种子、模型版本、指标口径和资源规格。

ARTIFACTS

数据与权重

不要直接提交大文件。使用对象存储、artifact registry、model registry、DVC 或 Git LFS 的受控路径。

算法 PR 必填信息

字段	必填内容	Reviewer 关注点
实验目的	要验证的假设，不是“尝试新模型”。	假设是否能被指标证伪。
数据口径	数据集版本、时间窗、过滤规则、泄漏防护。	训练 / 验证 / 测试是否污染。
指标	离线指标、线上指标、置信度或样本量。	是否只挑好看的单点结果。
复现命令	环境、配置、seed、硬件或近似资源。	别人是否能重跑关键结论。
上线风险	延迟、成本、召回率、偏置、回滚。	线上失败时能否降级。

HARD RULE

不要把真实用户数据、密钥、私有模型权重、商业数据快照、未脱敏日志或许可证不明数据提交到 Git。即使之后删除，历史里仍然可能存在。

15 | 恢复与排障：先定位根因，再修历史

Git 出错不要急着 reset
先保存证据

排障的专业顺序是：观察状态，保护当前工作，定位问题，选择最小恢复命令。不要把 `reset --hard` 当成万能修复。它会丢弃工作区和 index 里的改动，只有在明确要丢弃时才用。

撤回工作区改动

```
git restore path/to/file
git restore --staged path/to/file
```

找回历史位置

```
git reflog
git switch -c rescue <sha>
```

回滚已合并问题

```
git revert <bad-sha>
git revert -m 1 <merge-sha>
```

事故场景表

症状	先做什么	常用命令	不要做什么
误删文件但未提交	看 <code>git status</code> 。	<code>git restore <file></code>	不要全局 <code>hard reset</code> 。
commit 到错分支	记录 commit sha。	<code>git cherry-pick</code> 后回错分支 <code>reset</code>	不要直接复制文件。
push 后发现 bug	判断是否已合并。	未合并 <code>amend</code> / <code>rebase</code> ，已合并 <code>revert</code> 。	不要改写主干历史。
找不到丢失提交	查 <code>reflog</code> 。	<code>git reflog</code> ， <code>git branch rescue <sha></code>	不要继续清理 GC。
不知道哪个提交引入 bug	定义可重复测试。	<code>git bisect</code>	不要凭印象猜。

bisect 标准流程

```
git bisect start
git bisect bad HEAD
git bisect good <known-good-sha-or-tag>
./repro-test.sh
git bisect good # or: git bisect bad
git bisect reset
```

HUNT DISCIPLINE

同一个症状修了一次还在，说明假设不完整。重新从状态、日志、diff、测试和复现路径定位根因，不要连续堆补丁。

16 | 高级 Git 操作：只在知道代价时使用

高级命令不是炫技
是降低协作成本的工具

Git 的高级命令应服务于清晰历史、并行工作、精确回滚和跨分支移植。每个命令都有适用范围。团队应在文档里说明哪些操作允许、哪些需要 maintainer 审批。

命令	用途	安全边界	例子
<code>git worktree</code>	同一仓库多工作区并行开发。	避免 stash 混乱，适合 release hotfix。	<code>git worktree add ../repo-hotfix release/1.8</code>
<code>git cherry-pick</code>	把某个 commit 移植到另一分支。	适合 hotfix 回合主干或 release branch。	<code>git cherry-pick -x <sha></code>
<code>git stash</code>	临时保存未提交工作。	只作短期停车场，不作任务管理。	<code>git stash push -m "wip ranking"</code>
<code>git range-diff</code>	比较 rebase 前后 patch 系列。	stacked PR 或重写历史后给 reviewer 看。	<code>git range-diff old..new</code>
<code>git clean</code>	删除未跟踪文件。	危险，先 dry-run。	<code>git clean -nd</code> 后再 <code>-fd</code>
<code>git filter-repo</code>	重写仓库历史。	高风险，需团队公告和迁移窗口。	删除误提交大文件或密钥历史。

PATCH WORKFLOW

没有仓库权限或需要邮件协作时，可以用 `git format-patch` 和 `git am`。GitHub 场景下，优先 PR，因为 review、CI、讨论和合并状态都在同一处。

LFS / SUBMODULE

Git LFS 适合受控大二进制，submodule 适合把独立版本库挂入父项目。两者都会增加 contributor 成本，必须在 README 和 CI 中写清。

17 | 安全与合规: 协作流程必须默认防事故

安全不是发布前补丁
它在 Git 流程里

Git 历史具有传播性。密钥、用户数据、许可证不明代码和供应链配置一旦进入公开或共享历史，清理成本很高。安全规范必须放在开发前、PR 中和发布门禁里。

SECRETS

密钥

禁止提交 .env、token、私钥、cookie、生产配置。使用 secret scanning 和 pre-commit 检查。

SIGNING

签名

关键仓库启用 signed commits 或 signed tags，提升发布与来源可追溯性。

ACCESS

权限

最小权限、定期审计 collaborator、离职或外包结束立即移除访问。

SUPPLY CHAIN

依赖

新增依赖要说明必要性、许可证、维护状态、锁文件和安全扫描结果。

安全 PR 额外检查

变更类型	必须检查	推荐 reviewer
认证 / 授权	权限绕过、token 生命周期、失败默认行为、审计日志。	security + module owner。
命令执行 / 文件操作	路径穿越、shell 注入、symlink、dry-run、确认提示。	security + platform。
数据处理	PII、脱敏、保留期、导出范围、访问控制。	data owner + privacy。
CI / Actions	第三方 action pin、secret 暴露、fork PR 权限、OIDC。	infra / release engineering。
依赖变更	许可证、transitive deps、漏洞、维护者风险。	security 或 package owner。

INCIDENT RULE

密钥一旦提交到远端，不是删 commit 就结束。必须立刻吊销和轮换密钥，再清理历史，并检查日志中是否出现异常使用。

18 | AI 协作编码边界: 生成代码也要走工程门禁

AI 可以加速产出
不能替代责任归属

AI coding agent 能读代码、改文件、跑命令和生成 PR，但责任仍属于提交者和 reviewer。团队应把 AI 当成工程执行者，而不是绕过需求、设计、测试和 review 的理由。

INSTRUCTIONS

项目规则

在 AGENTS.md、CLAUDE.md 或等价文件中写清构建、测试、禁止区、提交规则和安全边界。

REVIEW

人类审查

AI 生成的 diff 仍按同等标准 review。尤其关注过度抽象、虚构 API、未运行测试和无关重构。

EVIDENCE

验证证据

要求 agent 在 PR 描述中列出实际运行的命令和失败项。未运行就不能写“已验证”。

AI PR 特殊风险

风险	表现	门禁
虚构 API	引用不存在函数、配置或依赖。	grep 或类型检查证明存在。
范围膨胀	顺手重构、格式化大面积文件。	PR scope 检查，拆分或退回。
验证幻觉	声称测试通过但没有命令输出。	PR 必填 verification output。
安全边界绕过	直接执行发布、删除、迁移、force push。	高风险命令人工确认。
上下文污染	把一次性事故报告写入长期规则。	只沉淀稳定约束，不沉淀临时事实。

OWNERSHIP

提交者必须能解释 AI 生成的每一处关键逻辑。不能解释的代码不应该合并，哪怕测试暂时是绿的。

19 | 角色、RACI 与团队制度

协作混乱常常不是工具问题
而是责任未定义

GitHub 的权限、review、issue、merge 和 release 都需要角色边界。团队应明确谁负责提出变更、谁拥有代码、谁能合并、谁负责发布、谁处理安全和外部贡献。

活动	Author	Reviewer	Code Owner	Maintainer	Release Manager	Security / Infra
Issue 分诊	C	I	C	A/R	I	C
实现与测试	A/R	C	C	I	I	C
PR 描述与验证	A/R	C	C	I	I	C
Code Review	C	A/R	A/R	C	I	C
Merge	C	C	C	A/R	I	I
Release	I	I	C	C	A/R	C
Security incident	I	I	C	C	C	A/R

R = Responsible, A = Accountable, C = Consulted, I = Informed。

团队制度最低集

Branch Policy

默认分支、保护规则、merge method、release branch、hotfix 分支。

Review SLA

普通 PR 首轮响应目标、紧急修复路径、卡住时升级机制。

Ownership Map

CODEOWNERS、模块 owner、领域 specialist 和替补机制。

Verification Contract

每类改动必须跑的测试、CI、人工验收和证据格式。

20 | 模板与 Checklist

模板的目的是
减少遗漏和来回询问

Bug Issue 模板

Symptom

Expected behavior

Reproduction

-
-
-

Environment

- Version:
- OS / runtime:
- Config:

Evidence

- Logs:
- Screenshot:
- Minimal repro:

Impact

- Severity:
- Workaround:

Feature Issue 模板

Problem

User / developer impact

Proposed behavior

Acceptance criteria

- []
- []

Non-goals

Risks

References

Review Checklist

维度	问题	硬阻塞示例
Scope	是否只解决声明问题。	混入无关重构或依赖升级。
Design	是否符合模块边界和现有模式。	绕过唯一数据源或引入循环依赖。
Correctness	边界、失败路径、并发、时序是否处理。	空输入、重试、超时、并发下错误。
Tests	测试是否能捕获本次行为变化。	逻辑变化无测试且无合理说明。
Docs	命令、配置、API、release notes 是否同步。	公共接口变更无文档。
Security	输入、权限、secret、依赖是否安全。	命令注入、密钥提交、权限绕过。

21 | 命令速查：高频与危险命令

先看作用层级
再敲命令

查看

```
git status -sb
git diff
git diff --cached
git log --oneline --graph -20
git show --stat <sha>
```

分支

```
git branch -vv
git switch main
git switch -c feat/123-x
git branch -d feat/123-x
git push origin --delete feat/123-x
```

暂存与提交

```
git add -p
git restore --staged <file>
git commit -m "fix(api): handle empty result"
git commit --amend
```

同步

```
git fetch origin
git pull --ff-only
git rebase origin/main
git merge --no-ff release/1.8
```

恢复

```
git restore <file>
git revert <sha>
git reflog
git reset --soft HEAD~1
```

PR with gh

```
gh issue list
gh pr create --draft
gh pr view --web
gh pr checks
gh pr merge --squash
```

危险命令红线

命令	风险	替代或保护
<code>git reset --hard</code>	丢弃工作区和暂存区改动。	先 <code>git status</code> ，必要时备份分支或 stash。
<code>git clean -fd</code>	删除未跟踪文件。	先 <code>git clean -nd</code> 预览。
<code>git push --force</code>	覆盖远端历史。	用 <code>--force-with-lease</code> ，仅限个人未合并分支。
<code>git filter-branch / filter-repo</code>	重写大量历史。	团队公告、备份、迁移计划。
删除远端分支 / tag	破坏他人引用和发布追溯。	确认 PR 已合并或 tag 未发布。

22 | 成熟度评分: 团队自检

评分不是为了好看
而是找下一步改进点

一个团队的 Git/GitHub 成熟度，不看使用了多少高级功能，而看变更是否小、主干是否稳、review 是否有效、发布是否可追溯、事故是否能快速恢复。

维度	0 到 2 分	3 到 4 分	5 分
分支纪律	main 直推，长期分支混乱。	大部分走 PR，但偶尔绕过。	短分支、保护主干、release 分支清晰。
Issue 质量	口头需求，issue 缺复现和验收。	有模板但执行不稳定。	intake、triage、labels、milestones 和 PR link 完整。
PR 可审查性	大 PR、混合改动、描述空。	多数 PR 可审查，偶有范围漂移。	小而完整，描述、验证、风险和回滚齐全。
Code Review	只看风格或只点 approve。	能发现问题，但标准不稳定。	设计、功能、复杂度、测试、安全、文档都有检查。
CI / 门禁	无自动化或 CI 常红。	有测试但覆盖不足。	required checks、merge queue 或等效机制保护主干。
发布 / 回滚	手工发布，无 tag 或 notes。	有 tag 和 release notes，但回滚不清。	SemVer、changelog、产物、回滚和 hotfix 路径可执行。
安全	密钥和权限靠自觉。	有扫描和权限管理。	secret、签名、依赖、CI 权限、安全披露都有制度。

0-15

先建立主干保护、PR 模板和最小 CI。不要急着引入复杂分支模型。

16-25

重点提升 PR 规模、review 标准、issue triage 和发布证据链。

26-35

进入工程效率优化：merge queue、owner map、自动 release notes、审查指标和培训机制。

23 | 来源与边界

公开资料可验证
内部制度不猜测

本手册不是任何公司的内部流程复刻。它只提炼公开材料中可迁移、可执行、可审查的工程原则，并落到 Git/GitHub 协作流程中。

主要来源

- Git 官方参考与 Pro Git 分支工作流。
- GitHub Docs: GitHub Flow、Issues、PR reviews、branch protection、merge queue、CODEOWNERS、Actions、contributing to a project。
- Google Engineering Practices: Code Review Standard、Small CLs、What to Look For、Handling Comments。
- Google Research: Modern Code Review, A Case Study at Google。
- Meta Engineering: Sapling source control、developer tools working at scale。
- OpenAI openai-python 与 OpenAI Cookbook 的 CONTRIBUTING 说明。
- CloudWeGo / ByteDance 开源项目 Kitex、Hertz 的 CONTRIBUTING 说明。
- Conventional Commits、Semantic Versioning、Keep a Changelog。

使用边界

- 本文给出的是标准流程和判断框架，具体命令应以目标仓库文档和 CI 为准。
- 涉及安全、隐私、发布、许可证、数据治理和合规时，组织制度优先于本文建议。
- 不同团队可调整 PR 大小、review 人数、merge method 和 release cadence，但不能取消主干健康、可验证和可回滚原则。
- 算法工程中的数据、模型权重和实验资产必须结合组织的数据平台、模型注册表和合规要求落地。

FINAL STANDARD

一次专业变更应能回答六个问题：为什么做、改了什么、如何验证、谁审查、如何合并、如何回滚。任何答不上来的环节，都是流程缺口。